K3

Yibo Ma

February 2025

1 Group

Sanjay Gupta — s347gupt — 20885923
Yibo Ma — y439ma — 20883952

2 Gitlab

Please locate the repository at: `https://git.uwaterloo.ca/s347gupt/CS452/-/tree/K3_IRQ_MERGE?ref_type=heads` and please use the following hash for testing the code: `b570e02e1afc95a4e592ab43cd2efc016b66abbd`.

3 Building and Running the Test

This section provides instructions on how to compile the kernel and execute the tests. To build the image, follow these steps:

1. Clone the repository from `git.uwaterloo.ca`.
2. Navigate to the `my_kernel` directory within the cloned repository:

   ```
   cd mykernel
   ```

3. Run the `make` command to build the kernel:

   ```
   make
   ```

   This command compiles the kernel and generates an image file (`iotest.img`) that can be run on the Raspberry Pi.

4 Kernel Structure

The Kernel project is organized within the git repo which includes the main kernel source code under my_kernel/. This directory contains separate subdirectories for assembly code (asm/), data structures (containers/), kernel core functionalities (kern/), and user-level tasks and servers (user/).

4.1 Classes/Algorithms

The code is organized into classes to enforce encapsulation and reduce coupling between separate processes.

4.1.1 Task Descriptor

Task descriptors isolate logic associated with typical functions needed by any running task; able to do the following:

- Create new tasks (setting themselves as the parent)
- Setting/Returning values associated with tasks (ex: returning TID)
- Each task descriptor contains a context structure: each is 64 bit int: SP, SPSR, ELR, and registers X0-X30. In order to save kernel state on context switches, the address of the context structure used by the kernel is passed into X0 and the task context structure goes to X1.

NOTE: Tasks are only issued to the kernel if memory is available for the task

4.1.2 Memory

Memory for tasks is allocated after the image, at 0x90000. Each stack is allocated 4 MB, and there can be a maximum of 64 tasks. The rationale behind this was that we have approximately 1 GB (minus 76 MB for videocore) after the end of image to work with, and having a task stack of size 256 MB is well within this region.

4.1.3 Kernel

- The kernel has several main private methods, such as Create, MyTid, MyParentTid, Yield, Exit, Send, Receive, Reply, AwaitEvent. These functions should only be called by the kernel's handler function (aside from bootstrapping in kernel constructor).
- The Kernel uses the memory class for allocating stacks to new tasks (called from create); a ring buffer for popping (create) and pushing (exit) available task ids; a pointer to the active task descriptor (last running task before entering kernel mode); an array of task descriptors (of size MAX_TASKS= 64), and a priority queue for O(1) retrieval of the task of highest priority.

- Scheduler: Pops the id of the highest priorty task from the ready queue, and returns a pointer to the corresponding entry in the task decriptor table.
- DispatchTask: Takes as input a task and sets it's state to active; calls kernel_to_task_asm assembly function to perform a context switch. See more details below.
- Handler: Takes as input the masked output of the ESR register (paramater 'N' passed in with svc call). Based on N, execute priveledged kernel operations (retrieving task information), creating new tasks, returning resources, etc.
- SetRetval: Save the system call return value into the task context X0 register (context.x[0]).

4.2 Assembly

4.2.1 kernel_to_task_asm

This is called by the DispatchTask function in order to perform a context switch.

- X0 contains address of kernel_context
- X1 contains address of task_context
- First we store x0 and x1 on the stack, note that sp points to the task_context
- Store all kernel registers, then store kernel SPSR, SP, ELR.
- Load pointer to task_context from stack
- load the registers and processor state
- eret to execute task in EL0

4.2.2 Vector Table

Upon issuing SVC calls, the architecture routes the exception to a vector table, defined in boot.S (boot.S stores the vector table in VBAR_EL1 system register). Each bank (4 exception handlers, 4 banks) routes to dummy routines refined in dummy.h. The one we are interested in is synchronous exceptions from lower EL, AArch64. This is defined in the table to branch to task_to_kernel_asm.

4.2.3 task_to_kernel_asm

- we load the task context pointer from the stack (taking care not to overwrite X0 and X1 (used in CREATE system call). We essentially perform the opposite of kernel_to_task_asm, saving the entire task state, popping the kernel context into X0 and then finally restoring the kernel context.

4.2.4 Cache Invalidation / Cleaning

- As seen later in the document, the cache functionality didn't seem to work (times didn't have much variability).
- We attempted to include code to invalidate the instruction cache and clean and invalidate the entire data cache by looping through each way/set.
- Issued ASM commands enabling the 2 and 12 bits for d/i caches, respectively.

4.3 Interrupt Management

Interrupt management is critical for the operation of our kernel, particularly for the clock server. The relevant code is implemented in `src/kern/kernel.cpp` and `src/kern/interrupts.cpp`.

4.3.1 Setup and Configuration

- **Generic Interrupt Controller (GIC)**: Initial setup is performed in `InitGIC()` within `interrupts.h`. This method configures the GIC distributor (`GICD_BASE`) and CPU interfaces to handle and route interrupts properly – this is done by writing 0x3 to GICD_CTLR and GICC_CTLR.
- **Enabling Timer Interrupts**: The kernel specifically enables the `TIMER_C1` interrupt, which is essential for the clock server's operation. This configuration allows the system to handle timer ticks accurately and efficiently.

4.3.2 Interrupt Handling

We reused the functionality for switching into the kernel from a task during interrupts as follows: IRQ's are routed to the vector table like "SVC" calls, so we defined a macro to act as a context switch into the kernel from a task, with a flag for if the switch is due to an interrupt (setting an appropriate return value based on the flag). The signals are then decoded in the main kernel loop and routed accordingly to the main handler (which then calls the IRQ handler).

- **IRQ Handler**: Located in `kernel.cpp`, this function processes incoming interrupts. On receiving a `TIMER_C1` interrupt, it triggers the clock server notification mechanism, which in turn checks for any tasks needing to be woken up.
- **AwaitEvent Implementation**: The `AwaitEvent()` system call is crucial for the clock notifier task, which uses it to wait for the next `TIMER_C1` tick. This system call blocks the task, reducing CPU usage until the next interrupt.

4.3.3 Clock Server Interaction

- **Clock Notifier Task**: Acts as a mediator between the hardware timer and the clock server. Upon receiving a tick through `AwaitEvent()`, it notifies the clock server, enabling it to update its internal tick count and manage task wake-ups.
- **Task Management**: The clock server, upon being notified of a tick, checks its priority queue for any tasks whose waiting time has expired and sends them wake-up messages (Based on the nature of SRR, issuing requests to the clock server places the client task in a blocked state until the server replies).

5 Data Structures

5.1 Queues

The code utilizes ring buffers to function as queues (FIFO). Ring buffer is defined using templates (in A0, they were used to pass commands to the Marklin hardware). For the purposes of K1, the ring buffer is used for storing ints (mapping between indices and array of task descriptors). As of K2, the ring buffer template class (deprecated) has been updated to the queue template class which support storing pointers for faster access. In K2, queues are used for managing incoming signup requests in the RPS server.

5.2 Priority Queues

Basic implementation (KISS) for min-heap priority queue, used exclusively by the kernel scheduler for determining which task to run. The class creates a static array (of size HEAP_SIZE), (since no heap), and updates a size field on push/pop operations. Similar to the Ring Buffer class, it is a template but implemented with ints (mapping).

6 User Space Algorithms

6.1 Nameserver

- Offers the following utility functions:

- `int WHOIS(const char *name)`: returns `tid` of task with corresponding name
- `int REGISTERAS(const char* name)`: registers the currently running task under the provided name

- When the FirstUserTask (FUT) runs, it will invoke a system call to create a high priority task that runs the name server - This is constantly in a while loop

receive state, so that it immediately gets `RECIEVE_BLOCKED`, only unblocking when other tasks invoke its global functions listed above.

6.2 Rock/Paper/Scissors Server

RPS Server is another service that is created by the FUT. RPS communicates with the name server to register under the name `RPSServer`. Similar to the name server, it indefinitely loops and puts itself in a `RECEIVE_BLOCKED` state. This allows `RPSClients` to communicate commands with it.

- Note that the RPS server uses queues to manage creating games with pairs of players.
- `RPSClients`: These are created by the FUT for the sole purpose of communicating with the RPS Server – the interface is defined so that the `moves` are encoded in a list, and having `QUIT` signals to the RPS Server that the registered player wants to quit.

6.3 Clock Server

The Clock Server in `src/user/clock_server.cpp` offers time-related functionalities to other tasks. It manages task delays and schedules using a priority queue, which ensures that tasks are woken up in the order of their scheduled wake times.

- `int TIME(int tid)`: Returns the current tick count since the clock server was initialized.
- `int DELAY(int tid, int ticks)`: Blocks the calling task for a specified number of ticks.
- `int DELAY_UNTIL(int tid, int ticks)`: Blocks the calling task until a specific tick count is reached.

The server operates in a continuous loop, processing delay and time requests from other tasks. It maintains a priority queue (`PQueue`), ordered by the earliest wake-up time, to efficiently manage requests.

6.3.1 Implementation and Operation

- Initialization: The FirstUserTask initializes the Clock Server and Name Server with high priorities to ensure they are processing data when possible. The clock server registers itself with the Name Server as "ClockServer" to facilitate discovery.
- SRR Mechanism: The server uses a send-receive-reply (SRR) mechanism to interact with tasks. Tasks send delay requests to the Clock Server, which then blocks them until their delay expires.

- Wake-up Process: On each tick (delivered by the Clock Notifier), the server checks the priority queue for any tasks whose wake time has passed. It then sends a reply to each of these tasks, unblocking them.

6.4 Clock Notifier

The Clock Notifier is a dedicated task created by the clock server to inform the server that the timer has ticked.

- **Operation Mode:** Operates in a continuous loop, awaiting `TIMER_TICK` events using the `AWAITEVENT` system call, which blocks the task until the timer interrupt is serviced by the kernel.
- **Tick Notification:** Upon a `TIMER_TICK` event, it constructs a `TICK` message and sends it to the Clock Server, ensuring that the server is promptly notified of the time progression.

6.4.1 Notifier's Role and Functionality

- **Essential Link:** Acts as a critical conduit between the hardware timer and the Clock Server, enabling precise delay management by ensuring the Clock Server is aware of each tick – after receiving the tick message, the server checks its priority queue to see if any delays have expired.

6.5 Clock Client Tasks

Clock Client Tasks are designed to test the functionality and responsiveness of the Clock Server within our kernel. Each client task, once created by the FirstUserTask, performs a sequence of operations to validate the time management capabilities of our system.

6.5.1 Implementation and Operation

- **Initialization**: Upon creation, each client task sends a message to its parent (the FirstUserTask) requesting specific parameters: a delay interval (t) in ticks, and a number (n) of delays.
- **Communication with Clock Server**: Utilizing the `WhoIs()` service, each client task retrieves the task identifier (TID) of the Clock Server to interact with it for timing services.
- **Delay Operations**: The client tasks enter a loop where they request a delay of t ticks, n times using the `Delay()` system call. After each delay, they log their TID, the delay interval, and the count of completed delays.
- **Output**: Outputs are printed on the console connected to the ARM box, providing real-time feedback on the task execution and timing accuracy.

7 UserTask

The FirstUserTask located in src/user/usertask.cpp initializes the system services and sets up the environment for testing the Clock Server.

7.1 Responsibilities and Implementation

- **Service Initialization**: It starts by creating essential system tasks such as the Name Server and Clock Server, ensuring they are up and running before any other operations.
- **Client Task Creation**: It then creates four client tasks with varying priorities. Each task is assigned different delay parameters.
- **Parameter Distribution**: After task creation, it uses the `Receive()` system call to collect initialization requests from each client task. It replies to each with the appropriate delay parameters:
  - Priority 3: Delay Interval = 10 ticks, Number of Delays = 20
  - Priority 4: Delay Interval = 23 ticks, Number of Delays = 9
  - Priority 5: Delay Interval = 33 ticks, Number of Delays = 6
  - Priority 6: Delay Interval = 71 ticks, Number of Delays = 3
- **Execution Monitoring**: Monitors the execution and ensures all client tasks are correctly initialized and operational.

7.2 Idle Task Involvement

In addition to managing client and server tasks, the FirstUserTask also initiates an Idle Task, which:

- Runs whenever other tasks are waiting or delayed, effectively using CPU downtime.
- Reports the proportion of the CPU time it consumes, serving as a diagnostic tool for assessing system performance and efficiency.
- Puts the CPU into a low-power state to conserve energy when not processing other tasks.